```
function
ps_spatio_temporal_consistency(Nifgs,Nps_atmo,Nps_defo,Npsc_selections,defo_mode
l_flag,stc_min_max,filt_mode,ts_noise_filter,ts_noise_filter_length)

% Function to calculate the spatio-temporal consistency
%
% Input:
%
% Output:
%
% ------------------------------------------------------------------------
% File............: ps_spatio_temporal_consistency.m
% Version & Date..: 1.7.2.16, 12-DEC-2009
% Author..........: Freek van Leijen
%                   Delft Institute of Earth Observation and Space Systems
%                   Delft University of Technology
% ------------------------------------------------------------------------
%
% This software is developed by Delft University of Technology and is
% intended for scientific use only. Applications for commercial use are
% prohibited.
%
% Copyright (c) 2004-2009 Delft University of Technology, The Netherlands
%
% Change log
%


% ------------------------------------------------------------------------
% Initialize
% ------------------------------------------------------------------------

global project_id az_spacing r_spacing Nlines Npixels fig orbit
global max_mem_buffer results_id Npar_max visible_plots

load cmap_defo

min_dist = stc_min_max(1); %[m], e.g. 50
max_dist = stc_min_max(2); %[m], e.g. 250
grid_size = max_dist;
buffer = 1;

switch defo_model_flag
  case 'yes'
    results_id = [{'atmo'};{'defo'}];
    Nps = [Nps_atmo Nps_defo];
    Nresults = 2;
  case 'no'
    results_id = {'atmo'};
    Nps = Nps_atmo;
    Nresults = 1;
end


% ------------------------------------------------------------------------
% Read background image
% ------------------------------------------------------------------------

delta_r = max(1,round(sqrt(Nlines*Npixels)/1000));
delta_az = max(1,delta_r*round(r_spacing/az_spacing));
Nlines_new = ceil(Nlines/delta_az);
Npixels_new = ceil(Npixels/delta_r);

mrm = repmat(uint8(NaN),Nlines_new,Npixels_new);
```

```
Nlines_buffer = min(Nlines,floor(max_mem_buffer/(8*Npixels)));
Nbuffer = ceil(Nlines/Nlines_buffer);
Nlines_rem_buffer = rem(Nlines,Nlines_buffer);

mrm_fid = fopen([project_id '_mrm_uint8.raw'],'r');
sl = 1;
el = 0;
for v = 1:Nbuffer
  if (v==Nbuffer)&&(Nlines_rem_buffer~=0)
    Nlines_buffer = Nlines_rem_buffer;
  end
  temp = fread(mrm_fid,[Npixels Nlines_buffer],'*uint8')';
  temp = temp(1:delta_az:end,1:delta_r:end);
  Nlines_buffer_new = size(temp,1);
  el = el+Nlines_buffer_new;
  mrm(sl:el,:) = temp;
  sl = sl+Nlines_buffer_new;
end
fclose(mrm_fid);

switch orbit
 case 'desc'
  mrm = flipud(fliplr(mrm));
end

fullscreen = [1 1 1280 1024];
%fullscreen=get(0,'Screensize');
[minratio,ratio_index] = min([fullscreen(3)/Npixels_new
fullscreen(4)/Nlines_new]);
if ratio_index(1)==1 %(1) to prevent multiple maxima
  figpos = [1 1 fullscreen(3) fullscreen(3)*Nlines_new/Npixels_new];
elseif ratio_index(1)==2
  figpos = [1 1 fullscreen(4)*Npixels_new/Nlines_new fullscreen(4)];
end


% --------------------------------------------------------------------
% Create grid
% --------------------------------------------------------------------

display('Creating grid ...');

grid_size_az = floor(grid_size/az_spacing);
grid_size_r = floor(grid_size/r_spacing);
grid_az = 1:grid_size_az:Nlines;
grid_r = 1:grid_size_r:Npixels;
if grid_az(end)~=Nlines
  grid_az = [grid_az Nlines+1];
else
  grid_az(end) = grid_az(end)+1;
end
if grid_r(end)~=Npixels
  grid_r = [grid_r Npixels+1];
else
  grid_r(end) = grid_r(end)+1;
end
Ngrid_az = length(grid_az)-1;
Ngrid_r = length(grid_r)-1;


for z = 1:Npsc_selections

  for w = 1:Nresults
```

```
display('Reading data ...');

% ----------------------------------------------------------------------
% Determine buffersize
% ----------------------------------------------------------------------

Nps_buffer = floor(max_mem_buffer/(5*(2*Nifgs+2)*8));

if (Nps_buffer>=Nps(z,w))
  Nps_buffer = Nps(z,w);
  Nbuffers = 1;
  Nps_rem_buffer = 0;
else
  Nbuffers = floor(Nps(z,w)/Nps_buffer);
  Nps_rem_buffer = rem(Nps(z,w),Nps_buffer);
  if (Nps_rem_buffer > 0)
    Nbuffers = Nbuffers + 1;
  end
end

Nps_buffer_orig = Nps_buffer;


% ----------------------------------------------------------------------
% Open files
% ----------------------------------------------------------------------

switch filt_mode
 case 'orig'
  ps_fid = fopen([project_id '_ps_results_' char(results_id(w)) '_sel' ...
              num2str(z) '.raw'],'r');
 case 'filt'
   ps_fid = fopen([project_id '_ps_results_filt_' ...
                   char(results_id(w)) '_sel' ...
               num2str(z) '_' ts_noise_filter '_length' ...
               num2str(ts_noise_filter_length) '.raw'],'r');
end

ps_az = repmat(NaN,Nps(z,w),1);
ps_r = repmat(NaN,Nps(z,w),1);

Nps_buffer = Nps_buffer_orig;
Nps_count = 0;

for v = 1:Nbuffers

  if (v == Nbuffers)&&(Nps_rem_buffer~=0)
    Nps_buffer = Nps_rem_buffer;
  end

  % ----------------------------------------------------------------------
  % Read data
  % ----------------------------------------------------------------------

  switch filt_mode
   case 'orig'
  ps_data = fread(ps_fid,[6+Npar_max+3*Nifgs Nps_buffer],'double')';

  ps_az(Nps_count+1:Nps_count+Nps_buffer) = ps_data(:,2);
  ps_r(Nps_count+1:Nps_count+Nps_buffer) = ps_data(:,3);

   case 'filt'
  ps_data = fread(ps_fid,[4+Nifgs Nps_buffer],'double')';
```

```
      ps_az(Nps_count+1:Nps_count+Nps_buffer) = ps_data(:,1);
      ps_r(Nps_count+1:Nps_count+Nps_buffer) = ps_data(:,2);
      end

      clear ps_data

      Nps_count = Nps_count+Nps_buffer;

    end



    % ---------------------------------------------------------------------
    % Creating index
    % ---------------------------------------------------------------------

    display('Creating index ...');

    az_index = repmat(NaN,Nps(z,w),1);
    r_index = repmat(NaN,Nps(z,w),1);

    for v = 1:Ngrid_az
      index = find(ps_az>=grid_az(v)&ps_az<grid_az(v+1));
      az_index(index) = v;
    end

    for v = 1:Ngrid_r
      index = find(ps_r>=grid_r(v)&ps_r<grid_r(v+1));
      r_index(index) = v;
    end


    % ---------------------------------------------------------------------
    % Calculate spatio-temporal consistency
    % ---------------------------------------------------------------------

    display('Calculating spatio-temporal consistency ...');

    ps_azx_orig = ps_az*az_spacing;
    ps_rx_orig = ps_r*r_spacing;

    stc = repmat(NaN,Nps(z,w),1);

    for g = 1:Ngrid_az

      display(['Computing spatio-temporal consistency of ' ...
               'azimuth grid ' num2str(g) 'of' num2str(Ngrid_az) '...']);

      index1 = find(az_index>=g-buffer&az_index<=g+buffer);
      Nindex = length(index1);

      if ~isempty(index1)

        ps_ts = repmat(NaN,Nindex,Nifgs);
        for k = 1:Nindex
        switch filt_mode
         case 'orig'
          fseek(ps_fid,((index1(k)-1)*(3*Nifgs+Npar_max+6)+
(Nifgs+Npar_max+6))*8,-1);
         case 'filt'
          fseek(ps_fid,((index1(k)-1)*(4+Nifgs)+(4))*8,-1);
        end
        ps_ts(k,:) = fread(ps_fid,[Nifgs 1],'double');
```

```
        end
        ps_azx = ps_azx_orig(index1);
        ps_rx = ps_rx_orig(index1);

        for h = 1:Ngrid_r
          index2 = find(r_index(index1)>=h-buffer&r_index(index1)<=h+buffer);
          index_orig1 = index1(index2);

          if ~isempty(index2)

            index3 = find(az_index(index_orig1)==g);
            index4 = find(r_index(index_orig1(index3))==h);
            index_orig2 = index2(index3(index4));
            index_orig3 = index1(index_orig2);
            Nps_buffer = length(index_orig3);

            if ~isempty(index_orig2)

              for v = 1:Nps_buffer

                daz_orig = ps_azx(index_orig2(v))-ps_azx(index2);
                dr_orig = ps_rx(index_orig2(v)) - ps_rx(index2);
                dist_orig = sqrt(daz_orig.^2+dr_orig.^2);
                dist_index = find(dist_orig>min_dist&dist_orig<max_dist);

                if ~isempty(dist_index)
                  dist = dist_orig(dist_index);
                  daz = daz_orig(dist_index);
                  dr = dr_orig(dist_index);
                  index = index2(dist_index);
                  Nps_buffer = length(index);

                  ts_sd = repmat(ps_ts(index_orig2(v),:),Nps_buffer,1) -
ps_ts(index,:);

                  ts_dd = ts_sd(:,1:end-1) - ts_sd(:,2:end);
                  std_ts_dd = std(ts_dd,0,2);
                  stc(index_orig3(v)) = min(std_ts_dd);
                end %if
              end %for v
            end %if
          end %if
        end %for h
      end %if
    end %for g


    % ---------------------------------------------------------------------
    % Write to file
    % ---------------------------------------------------------------------

    switch filt_mode
     case 'orig'
      stc_fid = fopen([project_id '_ps_stc_' char(results_id(w)) '_sel' ...
                   num2str(z) '.raw'],'w');
     case 'filt'
      stc_fid = fopen([project_id '_ps_stc_filt_' char(results_id(w)) '_sel' ...
                   num2str(z) '_' ts_noise_filter '_length' ...
                   num2str(ts_noise_filter_length) '.raw'],'w');
    end

    fwrite(stc_fid,stc','double');
    fclose(stc_fid);
```

```
    % ------------------------------------------------------------------------
    % Create plots
    % ------------------------------------------------------------------------

    delta2 = max(1,round(Nps(z,w)/10000));
    ps_az = ps_az(1:delta2:end);
    ps_r = ps_r(1:delta2:end);
    stc = stc(1:delta2:end);

    switch orbit
      case 'desc'
        ps_r = Npixels-ps_r+1;
        ps_az = Nlines-ps_az+1;
    end


    clim = [min(stc) max(stc)];
    cmap = cmap_defo;

    fig = fig + 1;
    figure(fig);hold on;
    if strcmp(visible_plots,'n')
      set(gcf,'visible','off');
    end

set(gcf,'position',figpos,'PaperPositionMode','auto','InvertHardcopy','off');

    subimage(mrm,gray(256));
    colormap(cmap);
    scatter(ps_r/delta_r,ps_az/delta_az,5,stc,'filled');
    caxis(clim);
    title('Spatio-temporal consistency','interpreter','none');
    xlabel('range');
    ylabel('azimuth');
    cbar_label = '[mm]';
    chandle = colorbar;
    set(get(chandle,'title'),'string',cbar_label);
    set(gca,'YDir','normal');
    axis off
    hold off

    switch filt_mode
     case 'orig'
      print('-dpng',['plots/' project_id '_stc_sel' num2str(z) '_'
char(results_id(w)) '.png']);
     case 'filt'
      print('-dpng',['plots/' project_id '_stc_filt_sel' num2str(z) '_'
char(results_id(w)) '_' ts_noise_filter '_length'
num2str(ts_noise_filter_length) '.png']);
    end

  end
end
```